Reinforcement Learning for Delta Hedging

Matthew Napoli (**mnapoli**), Parfait Kabore (**pkabore**), NY campus

2/24/2026

1 Introduction & Motivation

1.1 Introduction

Derivative contracts, whose value is derived from underlying assets, account for a large share of trading activity. In this paper, we consider a non-dividend-paying stock as the underlying asset. One common derivative on stocks is a European call option. A call gives the holder the right (but not the obligation) to buy the underlying asset at expiration time T for strike price K; a European call can only be exercised at maturity.

A common valuation approach for European calls is the Black-Scholes model. In this framework, we assume a risk-free interest rate r and constant volatility σ over the life of the option. The Black-Scholes call price is

$$C(S_t, t) = S_t\,\Phi(d_+) - Ke^{-r(T-t)}\,\Phi(d_-)$$

with

$$d_\pm = \frac{\ln(S_t/K) + \left(r \pm \frac{1}{2}\sigma^2\right)(T-t)}{\sigma\sqrt{T-t}}.$$

The value of an option changes with the current price of the underlying S_t. A measure one may be interested in computing is the option's **delta** (Δ_C), the approximate change in the option price for a small change in the underlying's price. In Black-Scholes, the call delta is

$$\Delta_C := \frac{\partial C}{\partial S_t} = \Phi(d_+).$$

This project applies reinforcement learning to hedging under transaction costs, using both simulated and real financial data. We implement the deep hedging setup of Cao et al. (2021), then extend it to stochastic transaction-cost regimes and real-world SPY/QQQ option-chain data. Our main finding is that RL-based hedging policies outperform simple delta-based benchmarks, although the strength of the conclusion is limited by computational budget and search coverage.

1.2 Motivation

Delta hedging is a form of risk-management which consists in buying and selling the underlying to maintain a **delta-neutral** position ($\Delta_C \approx 0$), such that small changes in the underlying's price do not affect the option's value. In the Black-Scholes framework, instantaneous delta-hedging assumes frictionless markets with no transaction costs. However, transaction costs in real-world discrete-market settings transform delta-hedging into a dynamic, sequential decision-making process that must account for the impact of rebalancing costs on portfolio performance. This setup naturally lends itself to reinforcement learning, since the problem can be framed as a Partially-Observable Markov Decision Process (PO-MDP) where an agent learns to optimize a trade-off between hedging behavior and total transaction costs by interacting with a stochastic environment.

1.3 Reinforcement Learning

Reinforcement learning (RL) is a framework in which an agent learns to make decisions by interacting with an environment and maximizing a cumulative reward signal. Instead of being told the correct action at each step, the agent learns effective behavior through trial-and-error and feedback from outcomes. A Reinforcement learning frameworks usually encompasses:

- **an environment**: here, it is the underlying asset's price dynamics and the type and specifications of the option contract
- **a state space**: here, relevant market information that the agent observes (current holdings, underlying price, time-to-maturity, transaction costs)
- **an action space**: here, the size and direction of the trade required to rebalance the portfolio
- **a reward function**: here, how well the hedge does each step, after trading costs

2 Methods

Our project follows the framework from *Deep Hedging of Derivatives Using Reinforcement Learning* by Cao, Chen, Hull, and Poulos (2021), with two extensions described in Section 3.

2.1 States, actions, & rewards

Following Cao's formulation, a trader holds a short position in a European call option and hedges by trading the underlying asset over n discrete time steps of length Δt. At each step i, the trader observes a state and chooses a hedge position H_{i+1} (shares of underlying to hold until the next step). Trading incurs proportional transaction costs at rate κ.

In the simulated environment, the stock price follows the classical Black–Scholes diffusion

$$dS_t = rS_t\,dt + \sigma S_t\,d\widetilde{W}_t,$$

where we set $r = 0$ and $\sigma > 0$ is constant over the life of the option.

At each trading time i, the agent observes a **state** S_i, selects an **action** A_i, and receives a **reward** R_{i+1}. The state summarizes the information used for decision-making, the action specifies the hedge adjustment, and the reward measures hedging performance (including transaction costs).

- State at time $i\Delta t$: $S_i = (H_i, S_i, \tau_i)$, where H_i is the current hedge, S_i is spot price, and $\tau_i = T - i\Delta t$ is time to maturity.
- Action: choose $H_{i+1} \in [H_{\min}, H_{\max}]$.
- Reward (accounting P&L formulation):

$$R_{i+1} = \underbrace{(V_{i+1} - V_i)}_{\text{change in option value}} + \underbrace{H_i\,(S_{i+1} - S_i)}_{\text{hedge P\&L}} - \underbrace{\kappa_i\,|S_{i+1}\,(H_{i+1} - H_i)|}_{\text{transaction cost}} - \underbrace{\lambda(H_i - \Delta_i)^j}_{\text{delta overexposure cost}}$$

  with setup cost $-\kappa_0\,|S_0 H_0|$ and terminal liquidation cost $-\kappa_n\,|S_n H_n|$.

The option value $V_i = -(\#\text{ options}) \times \text{BSCallPrice}(S_i, \tau_i)$ uses a valuation volatility σ_{val} that may differ from the simulation volatility. This is the "hybrid approach" in Cao et al. (2021).

2.2 Optimization Target

Our goal is to find a hedging policy π that minimizes

$$Y(0) = \mathbb{E}\left[C_0\right] + c\sqrt{\mathbb{E}\left[C_0^2\right] - \mathbb{E}\left[C_0\right]^2} \tag{2}$$

where

$$C_0 := -\sum_{i=1}^{T} \gamma_i R_i$$

is the total hedging cost from time 0 onward, c is a risk-aversion parameter (set to 1.5 throughout), and $\gamma_i \in [0, 1]$ are discount factors (set to 1 throughout to weigh all future rewards equally with immediate ones and maximize the total sum of rewards).

Note: Because we evaluate hedging policies via cost (not reward), **lower** values of average cost (Avg Cost), standard deviation of cost (Std Cost), and the Y-metric indicate better hedging performance unless otherwise stated.

2.3 Double Q-functions

Standard RL maximizes expected cumulative reward. The Q-function $Q(x, a)$ gives the expected cumulative reward obtained by taking action a in state x and thereafter following a given policy. Cao et al. extend this framework to handle the mean–standard-deviation objective in (2) by tracking two Q-functions:

- $Q_1(x, a) \approx \mathbb{E}[G \mid x, a]$ (expected future reward),
- $Q_2(x, a) \approx \mathbb{E}\left[G^2 \mid x, a\right]$ (expected squared future reward),

where G denotes cumulative discounted reward from the current state-action pair onward. The greedy action minimizes

$$F(x, a) = Q_1(x, a) + c\sqrt{Q_2(x, a) - Q_1(x, a)^2}. \tag{3}$$

Q_1 target (standard Bellman):

$$y_1 = r + \gamma(1 - d)Q_1^{\text{tgt}}(x', a')$$

Q_2 target (second-moment Bellman):

$$y_2 = r^2 + 2\gamma r(1 - d)Q_1^{\text{tgt}}(x', a') + \gamma^2(1 - d)Q_2^{\text{tgt}}(x', a')$$

These targets follow from expanding $\mathbb{E}\left[(r + \gamma G')^2\right]$ and using the definition of Q_2.

2.4 Deep Deterministic Policy Gradient (DDPG)

Q-learning is a model-free reinforcement learning algorithm that iteratively estimates the optimal Q-function by updating $Q(s, a)$ using observed rewards. Because the state and action spaces are continuous (hedge position is a real number), tabular Q-learning is impractical. Cao et al. therefore use a DDPG-style actor-critic method:

- Actor $\pi(s; \theta)$: neural network representing a deterministic policy, mapping states to continuous hedge positions.
- Critics $Q_1(x, a)$ and $Q_2(x, a)$: neural networks estimating the first and second moments of cumulative reward.
- The actor is updated via the deterministic policy gradient by minimizing the mean–standard-deviation objective

$$F(x, \pi(s)) = Q_1(x, \pi(s)) + c\sqrt{Q_2(x, \pi(s)) - Q_1(x, \pi(s))^2}.$$

- Each critic is trained by minimizing mean squared error against its Bellman target:

$$y_1 = r + \gamma(1 - d)Q_1^{\text{tgt}}(x', a'), \quad y_2 = r^2 + 2\gamma r(1 - d)Q_1^{\text{tgt}}(x', a') + \gamma^2(1 - d)Q_2^{\text{tgt}}(x', a').$$

- Target networks (soft-updated copies) reduce instability by slowing parameter updates.
- Experience replay improves sample efficiency and reduces temporal correlation.
- Exploration is induced by Gaussian noise added to actor outputs during training.

2.5 Experimental Setup and Validation Protocol

To make the empirical pipeline reproducible, we summarize the selection/evaluation process used in this report.

1. **Simulation hyperparameter search (selection stage).** We evaluate a set of hyperparameter configurations in a fixed simulated environment (3-month maturity, daily rebalancing) with constant transaction cost $\kappa = 0.01$, and rank configurations by the Y-metric (again, lower is better).

2. **Simulation transfer checks.** We take the top 25 configurations from the $\kappa = 0.01$ search and re-evaluate them in two settings: (i) constant $\kappa = 0.05$ and (ii) stochastic transaction costs (Markov-chain regime model).

3. **Final candidate retraining.** We select the top 3 candidate RL models based on cross-setting performance and retrain them for longer (1000 episodes) before comparing them to baselines.

4. **Real-data evaluation.** We train/evaluate the selected RL candidates and baselines on SPY/QQQ option-chain episodes after preprocessing and a time-based split by expiration date.

3 Extensions Beyond Cao et al.

A significant portion of the project was devoted to implementing the Cao et al. simulated Brownian-motion environment, along with the replay buffer and RL agents. After implementing the core framework, we added two extensions and one scope restriction:

1. **Stochastic transaction costs.** Instead of a fixed κ, we model a 3-state Markov chain for liquidity regimes: State 0 ("abundant," $\kappa = 0.002 = 20$ bps), State 1 ("normal," $\kappa = 0.01 = 100$ bps), State 2 ("stressed," $\kappa = 0.025 = 250$ bps)

   Transitions follow a persistent Markov chain with default transition matrix

   $$P = \begin{pmatrix} 0.93 & 0.07 & 0.00 \\ 0.04 & 0.92 & 0.04 \\ 0.00 & 0.10 & 0.90 \end{pmatrix}.$$

   The current transaction-cost state κ_i is added to the state vector so the agent can condition its hedge on the liquidity regime. The state vector then becomes 4-dimensional $[H_i, S_i, \tau_i, \kappa_i]$ instead of the paper's 3-dimensional state $[H_i, S_i, \tau_i]$.

2. **Hyperparameter search and model selection.** wWe hypothesized that the simulated environment (with fixed horizon and daily hedging) could support a systematic search for stronger learning parameters. Because the joint space of environment choices and model hyperparameters is large, an exhaustive search is basically impossible. So we use a staged selection procedure to balance computational tractability and robustness. One scope restriction is that our simulated environment is fixed at a 3-month maturity with daily rebalancing, which is computationally manageable while still providing a good hedging problem. Our procedure:

   (a) We first search over a range of hyperparameters in a simulated environment with constant $\kappa = 0.01$.

   (b) Then we take the top 25 hyperparameter settings and test their transfer performance when (i) $\kappa = 0.05$ and (ii) transaction costs are stochastic, using the Markov chain P above.

   (c) Lastly, we select the top 3 model candidates, train them longer, and compare them against two baselines: full delta hedging, and delta-with-bands (only rebalance when $|H_{\text{target}} - H_{\text{curr}}| > b$).

3. **Application to real-world data.** After selecting learning parameters in simulation, we compare the RL policy to baseline hedging policies on real data using the Kaggle datasets "$SPY Option Chains - Q1 2020 - Q4 2022" and "$QQQ Option Chains - Q1 2020 to Q4 2022." Our preprocessing pipeline is:

(a) Remove extraneous columns and rows with missing values. Instead of the last traded price (which can be stale, especially for far OTM/ITM options), use a volume-weighted average price (VWAP)-based price proxy.

(b) Group data into episodes by strike and expiration date.

(c) Filter out episodes with fewer than 25 time steps.

(d) Split episodes into train / validation / test sets using expiration date (time-based split to reduce look-ahead bias).

(e) Feed episodes into a modified version of the RL framework for training and out-of-sample evaluation.

4 Results

We evaluate model performance using Avg Cost, Std Cost, and the Y-metric defined in (1). Once again, because these are cost-based metrics, **lower values indicate better hedging performance**. We compare RL models to full delta hedging and a delta-with-bands benchmark.

The first result we noticed in simulation is that the difference between stochastic transaction costs and deterministic constant transaction costs did not produce a dramatic change in the ranking of top-performing models. In particular, the top 25 models selected under constant $\kappa = 0.01$ generalized reasonably well to the stochastic transaction-cost regime, as shown in Figure 1.

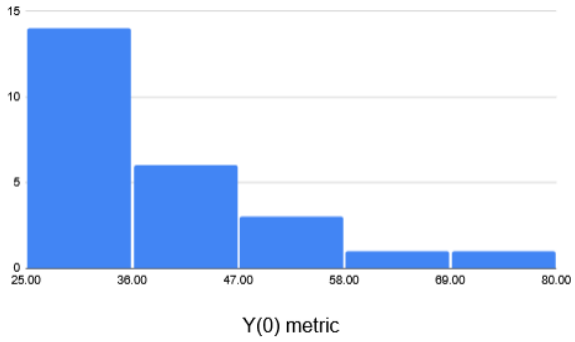

Figure 1: Generalization of the top 25 RL models (selected under constant $\kappa = 0.01$) to alternative transaction-cost regimes. Lower plotted performance metrics indicate better hedging performance.

Figure 1 above suggests that most selected models maintain acceptable performance under stochastic transaction costs. By contrast, transfer to a uniformly higher constant transaction-cost regime ($\kappa = 0.05$) is weaker for several models.

Figure 2 below shows the full comparison table for the top 25 models across environment settings.

Agent-Hyperparameters						Constant 0.01			Constant 0.05			Stochastic Transaction Cost			
Tau	Actor Learning Rate	Critic Learning Rate	noiseStd	noiseClip	RewardScalar	Avg	Std	Y(0)	Avg	Std	Y(0)	Avg	Std	Y(0)	Average Y(0)
0.0001	0.0001	0.00024	0.025	0.18	1	12.7	12.2	**31.0**	-0.2	24.7	**36.9**	0.1	19.3	29.1	**32.3**
0.0001	0.0001	0.00004	0.045	0.10	1	8.4	15.0	**30.9**	-0.2	24.7	**36.9**	-0.2	20.5	30.6	**32.8**
0.0001	0.0001	0.00024	0.025	0.02	1	6.5	17.1	**32.1**	-0.2	24.7	**36.9**	-0.2	20.0	29.8	**32.9**
0.0006	0.0001	0.00024	0.045	0.18	1	12.2	13.0	**31.6**	-0.2	24.7	**36.9**	0.1	20.2	30.4	**33.0**
0.0006	0.0001	0.00004	0.045	0.02	1	12.9	13.6	**33.3**	-0.2	24.7	**36.9**	-0.2	19.7	29.3	**33.2**
0.0001	0.0001	0.00004	0.005	0.02	1	10.7	13.9	**31.6**	-0.2	24.7	**36.9**	9.2	15.0	31.7	**33.4**
0.0006	0.0006	0.00024	0.025	0.02	1	11.5	12.9	**30.9**	-0.2	24.7	**36.9**	-1.0	22.8	33.2	**33.7**
0.0001	0.0001	0.00024	0.045	0.18	1	9.6	14.0	**30.6**	-0.2	24.7	**36.9**	9.6	16.0	33.6	**33.7**
0.0006	0.0001	0.00004	0.025	0.02	1	12.1	12.8	**31.2**	-0.2	24.7	**36.9**	-1.0	22.8	33.2	**33.8**
0.0006	0.0006	0.00004	0.025	0.18	50	14.5	12.5	**33.3**	-0.2	24.7	**36.9**	0.7	21.5	32.9	**34.4**
0.0006	0.0006	0.00024	0.005	0.18	1	13.0	13.3	**33.0**	-0.2	24.7	**36.9**	-1.0	22.8	33.2	**34.4**
0.0006	0.0001	0.00024	0.045	0.10	1	16.0	11.9	**33.8**	-0.2	24.7	**36.9**	10.7	16.2	35.0	**35.3**
0.0001	0.0006	0.00024	0.045	0.02	1	5.4	17.7	**31.9**	-0.2	24.7	**36.9**	12.4	17.7	38.9	**35.9**
0.0006	0.0001	0.00004	0.005	0.02	1	10.1	14.3	**31.6**	-0.2	24.7	**36.9**	7.5	23.4	42.5	**37.0**
0.0006	0.0006	0.00004	0.025	0.02	1	9.9	13.4	**30.1**	-0.2	24.7	**36.9**	15.7	19.2	44.5	**37.2**
0.0001	0.0006	0.00024	0.005	0.18	1	9.2	16.6	**34.0**	-0.2	24.7	**36.9**	10.5	20.8	41.8	**37.6**
0.0006	0.0006	0.00024	0.045	0.02	1	14.5	12.9	**33.9**	-0.2	24.7	**36.9**	15.3	19.2	44.2	**38.3**
0.0006	0.0001	0.00004	0.045	0.10	1	9.7	13.8	**30.4**	-0.2	24.7	**36.9**	19.6	22.6	53.5	**40.3**
0.0001	0.0006	0.00024	0.005	0.02	1	10.2	14.5	**31.9**	-0.2	24.7	**36.9**	18.8	22.5	52.4	**40.4**
0.0001	0.0001	0.00004	0.045	0.02	1	7.3	16.2	**31.6**	-0.2	24.7	**36.9**	20.3	26.8	60.5	**43.0**
0.0001	0.0006	0.00004	0.025	0.10	1	9.2	14.8	**31.3**	191.6	35.2	**244.4**	-0.6	19.9	29.2	**101.7**
0.0001	0.0006	0.00004	0.045	0.10	1	12.5	14.4	**34.1**	191.6	35.2	**244.4**	7.2	17.3	33.1	**103.9**
0.0006	0.0006	0.00024	0.045	0.10	1	5.0	17.8	**31.7**	191.6	35.2	**244.4**	14.8	19.3	43.7	**106.6**
0.0006	0.0006	0.00004	0.005	0.10	1	10.0	14.6	**31.9**	191.6	35.2	**244.4**	18.3	20.2	48.5	**108.3**
0.0001	0.0006	0.00024	0.045	0.10	1	10.0	14.2	**31.3**	191.6	35.2	**244.4**	29.5	31.2	76.4	**117.4**

Figure 2: Top-25 hyperparameter configurations ranked under constant $\kappa = 0.01$, with transfer performance reported under constant $\kappa = 0.05$ and stochastic transaction-cost regimes. (If space allows, increasing font size or moving the full table to an appendix would improve readability.)

After retraining the final model candidates for 1000 episodes in the simulated environment (instead of the shorter initial training budget used in the search stage), the selected RL models produce consistent results and outperform the benchmark strategies in our simulated tests.

	Constant 0.01			Constant 0.05			Stochastic Transaction Cost			
	Avg Cost	Std Cost	Std Cost	Avg Cost	Std Cost	Y Metric	Avg Cost	Std Cost	Y Metric	Avg Y Metric
Model #1	15.6	12.8	**34.7**	-0.2	24.7	**36.9**	-0.8	21.6	**31.6**	**34.4**
Model #2	16.5	12.9	**35.8**	-0.2	24.7	**36.9**	11.0	16.9	**36.3**	**36.3**
Model #3	17.5	28.1	**59.7**	-0.2	24.7	**36.9**	12.0	17.7	**38.5**	**45.0**
DeltaWithBands	14.0	11.7	**31.6**	70.3	49.2	**144.2**	13.8	18.8	**42.0**	**72.6**
Delta	15.3	13.5	**35.6**	77.1	65.8	**175.8**	15.0	21.9	**47.8**	**86.4**

Figure 3: Performance of final RL model candidates after longer training (1000 episodes) in the simulated environment, compared with delta-hedging baselines. Lower values of Avg Cost / Std Cost / Y indicate better performance.

Lastly, we train and evaluate the selected models on real-world SPY/QQQ option-chain data (with $\kappa = 0.05$ in the evaluation setup). In our experiments, the RL candidates substantially outperform the baseline hedging strategies and appear to converge to very similar policies. The agents also achieve positive average reward/P&L under our accounting convention, which was surprising given our simulation setup choices (e.g., ATM-focused simulated episodes). We can't interpret positive average reward as evidence of predictive alpha; it may just reflect the reward/accounting convention, sample composition, filtering choices, or data-quality effects. Figure 4 below reports the real-data comparison table.

	SPY			QQQ			
	Avg Cost	Std Cost	Y Metric	Avg Cost	Std Cost	Y Metric	Avg Y Metric
Model #1	-29.2	90.7	**106.9**	-43.2	94.0	**97.8**	**102.4**
Model #2	-29.2	90.7	**106.9**	-43.2	94.0	**97.8**	**102.4**
Model #3	-29.2	90.7	**106.9**	-43.2	94.0	**97.8**	**102.4**
DeltaWithBands	169.9	442.0	**832.9**	133.2	331.8	**630.9**	**731.9**
Delta	632.0	1010.6	**2147.9**	355.0	554.6	**1186.8**	**1667.4**

Figure 4: Real-data performance on SPY/QQQ option-chain episodes. RL candidates are compared with full delta and delta-with-bands baselines using cost-based metrics (lower is better).

To compare whether the top RL candidates learned materially different policies, we also inspect their reward trajectories on held-out data. Figure 5 shows reward trajectories for two candidate

models on real-world testing data. The trajectories are very similar, suggesting convergence toward similar effective policies under the tested configuration.

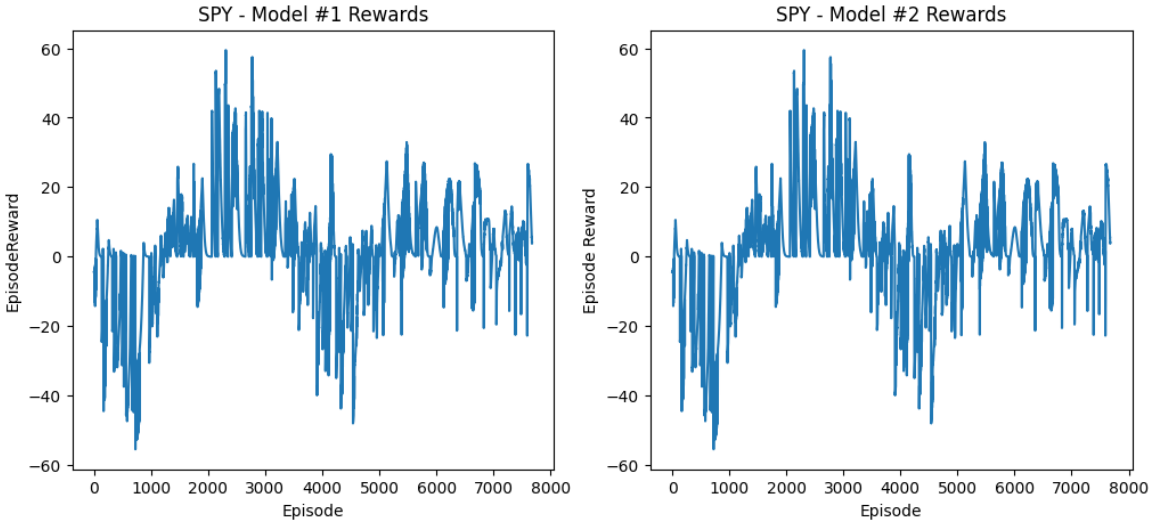

Figure 5: Reward trajectories for Model #1 and Model #2 on real-world testing data. The near-overlap supports the interpretation that the top candidate policies converge to similar behavior.

5 Discussion, Limitations, & Reflection

5.1 Discussion

A major practical lesson from this project is the size of the design space: environment choices, simulation assumptions, agent hyperparameters, and real-data preprocessing decisions all affect outcomes. Although we initially considered 200+ hyperparameter configurations, computational constraints limited how much of the broad parameter space we could explore. Multi-hour runtime for RL training and retraining forced us to prioritize staged selection and transfer-based filtering rather than an exhaustive search.

Another early challenge was numerical stability associated with the second critic Q_2, which tracks squared rewards. The magnitude of Q_2 was sometimes difficult to control (Figure 6), which motivated a more systematic hyperparameter search rather than ad hoc tuning. In the end, this approach produced candidates that transferred well across multiple simulated environments and performed strongly on our real-data tests.

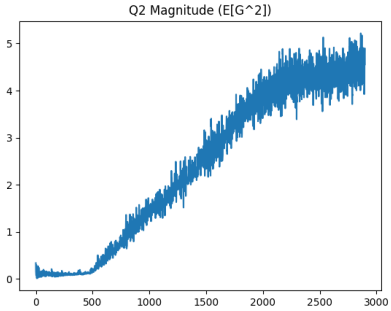

Figure 6: Q_2 magnitude instability / non-convergence example. This motivated the staged hyperparameter-search and transfer-evaluation procedure used in the project.

5.2 Limitations

Our conclusions should be interpreted with the awareness of several limitations:

- Training and search are expensive, limiting hyperparameter coverage and the number of robustness checks (e.g., across random seeds).
- Most of our simulation experiments use a 3-month option with daily rebalancing, so conclusions may not transfer to very short- or long-horizon hedging frequencies without further study.
- Real-data results may be sensitive to filtering choices, pricing proxies (e.g., VWAP vs. last trade), and episode construction rules.
- Our transaction-cost and market-impact assumptions are simplified relative to real execution conditions.

5.3 Conclusion and Reflection

Despite computational constraints and a large design space, we produced a functioning implementation of the Cao et al. deep hedging framework with two meaningful extensions: stochastic transaction-cost regimes and real-data evaluation on SPY/QQQ option chains. In our experiments, the RL candidates outperformed delta-based baselines and generalized reasonably well from the base simulated setting to the stochastic transaction-cost regime. At the same time, variability across alternative simulated regimes and the sensitivity of Q_2 dynamics emphasize a need for broader robustness testing before drawing stronger conclusions. For potential future iterations/extensions of this project, we would prioritize the following extensions:

- **Parallel and batched training.** Parallelizing episode generation and agent training could substantially reduce compute time and allow a much broader search over environment and hyperparameter configurations. Vectorized environments (batched episode simulation) could further improve sample throughput and training stability.
- **Broader hedging horizons.** We would test whether the learned policies transfer across hedging frequencies (e.g., intraday, daily, weekly, monthly), since compute limitations constrained most simulated experiments to daily hedging over a 3-month horizon.
- **More realistic market-impact modeling.**
  - Develop a more realistic transaction-cost function informed by limit-order-book structure and trade size.
  - Extend the environment with temporary/permanent market-impact effects and penalties for manipulative trading behavior.
- **Stronger bias controls and deployment-style evaluation.** We already use a time-based train/test split to reduce look-ahead bias. A next step is a simulated paper-trading or rolling-window evaluation setup to further test for implementation leakage and time-series bias.

5.4 Artificial Intelligence

We used Artificial Intelligence to debug sections of this project, and to draft the skeleton of this report.

References

[1] Cao, J., Chen, J., Hull, J., & Poulos, Z. (2021). Deep Hedging of Derivatives Using Reinforcement Learning. *Journal of Financial Data Science*, 3(1), 10–27.

[2] Lillicrap, T. P., Hunt, J. J., Pritzel, A., Heess, N., Erez, T., Tassa, Y., Silver, D., & Wierstra, D. (2016). Continuous control with deep reinforcement learning. In *International Conference on Learning Representations (ICLR)*.

[3] Kaggle Dataset: "$SPY Option Chains - Q1 2020 - Q4 2022". Accessed February 2026.

[4] Kaggle Dataset: "$QQQ Option Chains - Q1 2020 to Q4 2022". Accessed February 2026.